

Nguyên Lý Pólya Và Ứng Dụng

Fu Wenjie

2001

Nguồn gốc: Pólya principle and applications

IOI Candidate Team Papers/2001/Fu Wenjie.pdf

Abstract

Nguyên lý Pólya là một công cụ hiệu quả và gọn trong tổ hợp, dùng để đếm số trạng thái tổ hợp khác nhau về bản chất khi có các phép đối xứng. Bài viết giới thiệu các khái niệm nhóm, hoán vị, nhóm hoán vị, bổ đề Burnside, định lý Pólya, rồi áp dụng vào bài toán tô màu hình vuông dưới phép quay.

1 Ví dụ mở đầu

Ví dụ 1. Tô màu một bảng 2×2 bằng hai màu đen và trắng. Hỏi có thể thu được bao nhiêu hình khác nhau? Hai phương án được xem là cùng một phương án nếu có thể quay để trùng nhau.

Vì quy mô bài toán rất nhỏ, ta có thể liệt kê toàn bộ các phương án tô màu. Trong bản gốc, mười sáu cách tô được ký hiệu là $C_1, \dots, C_{16}$. Có bốn phép quay của bảng 2×2 : quay 0° , 90° , 180° , và 270° . Trong bài viết này, phép quay mặc định là quay theo chiều kim đồng hồ.

Sau khi thử, ta thấy chỉ có 6 phương án khác nhau về bản chất:

$$\{C_1\}, \quad \{C_2\}, \quad \{C_3, C_4, C_5, C_6\}, \quad \{C_7, C_8, C_9, C_{10}\}, \quad \{C_{11}, C_{12}\}, \quad \{C_{13}, C_{14}, C_{15}, C_{16}\}.$$

Nói cách khác, các phần tử trong cùng một tập trên có thể biến đổi qua lại bằng phép quay và phải được tính là một phương án.

Để minh họa cách đánh số, ta dùng 0 cho trắng và 1 cho đen, đọc bốn ô theo thứ tự trái trên, phải trên, trái dưới, phải dưới. Khi đó mười sáu cách tô có thể xem là các xâu nhị phân từ 0000 tới 1111. Hình trong bản gốc liệt kê các hình này và sáu lớp tương đương; ở đây ta dùng mô tả bằng ký hiệu vì hình không còn khôi phục được từ văn bản trích xuất.

Khi bài toán đổi từ bảng 2×2 thành bảng 20×20 , thậm chí 200×200 , ta không thể liệt kê từng phương án nữa. Khi đó nguyên lý Pólya trở thành một cách giải rất tốt. Trước khi đi vào nguyên lý Pólya, ta cần nhắc lại một vài khái niệm.

2 Nhóm, hoán vị và nhóm hoán vị

Nhóm. Cho một tập $G = \{a, b, c, \dots\}$ và một phép toán hai ngôi trên G . Nếu thỏa mãn bốn điều kiện sau:

1. **Đóng:** với mọi $a, b \in G$, tồn tại $c \in G$ sao cho $a * b = c$.

2. **Kết hợp:** với mọi $a, b, c \in G$,

$$(a * b) * c = a * (b * c).$$

3. **Phần tử đơn vị:** tồn tại $e \in G$ sao cho với mọi $a \in G$,

$$a * e = e * a = a.$$

4. **Phần tử nghịch đảo:** với mọi $a \in G$, tồn tại $b \in G$ sao cho

$$a * b = b * a = e.$$

Ta ký hiệu $b = a^{-1}$.

thì G được gọi là một nhóm dưới phép toán $*$. Thường viết gọn $a * b$ thành ab .

Hoán vị. Một hoán vị trên n phần tử $1, 2, \dots, n$ được viết

$$\begin{pmatrix} 1 & 2 & \cdots & n \\ a_1 & a_2 & \cdots & a_n \end{pmatrix},$$

nghĩa là 1 được thay bởi một số a_1 trong $1..n$, 2 được thay bởi một số a_2 , và cứ thế tới n , trong đó $a_1, a_2, \dots, a_n$ đôi một khác nhau.

Trong ví dụ tô màu 2×2 , xét mười sáu phương án $C_1, \dots, C_{16}$ như các phần tử đang bị tác động bởi phép quay. Bốn phép quay tạo ra bốn hoán vị sau:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a_1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a_2	1	2	6	3	4	5	10	7	8	9	12	11	16	13	14	15
a_3	1	2	5	6	3	4	9	10	7	8	11	12	15	16	13	14
a_4	1	2	4	5	6	3	8	9	10	7	12	11	14	15	16	13

Ở đây a_1, a_2, a_3, a_4 lần lượt là quay $0^\circ, 90^\circ, 180^\circ$ và 270° .

Nhóm hoán vị. Nhóm hoán vị là một nhóm mà phần tử là các hoán vị, còn phép toán là phép nối hoán vị. Ví dụ:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 3 & 1 \end{pmatrix}.$$

Có thể kiểm tra rằng nhóm hoán vị thỏa mãn bốn điều kiện của nhóm. Trong ví dụ đang xét,

$$G = \{\text{quay } 0^\circ, \text{quay } 90^\circ, \text{quay } 180^\circ, \text{quay } 270^\circ\}.$$

3 Bổ đề Burnside

Ta xét công thức

$$|E_k| \cdot |Z_k| = |G|, \quad k = 1, \dots, n.$$

Công thức này liên quan trực tiếp tới số phần tử của nhóm và rất hữu ích.

Lớp hoán vị giữ cố định K . Giả sử G là một nhóm hoán vị trên $1..n$. Nếu K là một phần tử trong $1..n$, tập tất cả các hoán vị trong G làm K không đổi được ký hiệu là Z_K , gọi là lớp hoán vị trong G giữ cố định K .

Trong ví dụ này, G là nhóm hoán vị trên các phương án tô màu $1, \dots, 16$. Với phương án 1, cả bốn hoán vị đều giữ nó không đổi, nên

$$Z_1 = \{a_1, a_2, a_3, a_4\}.$$

Với phương án 3, chỉ a_1 giữ nó không đổi, nên $Z_3 = \{a_1\}$. Với phương án 11, hai hoán vị a_1 và a_3 giữ nó không đổi, nên

$$Z_{11} = \{a_1, a_3\}.$$

Lớp tương đương. Giả sử G là một nhóm hoán vị trên $1..n$. Nếu K là một phần tử trong $1..n$, quỹ đạo của K dưới tác động của G được ký hiệu là E_K . Đó là tập tất cả phần tử mà K có thể biến thành khi áp dụng các hoán vị trong G .

Trong ví dụ:

$$E_1 = \{1\}, \quad E_3 = \{3, 4, 5, 6\}, \quad E_{11} = \{11, 12\}.$$

Các dữ liệu này khớp hoàn toàn với công thức:

$$|E_1||Z_1| = 1 \cdot 4 = 4 = |G|,$$

$$|E_3||Z_3| = 4 \cdot 1 = 4 = |G|,$$

$$|E_{11}||Z_{11}| = 2 \cdot 2 = 4 = |G|.$$

Do giới hạn dung lượng, bản gốc không chứng minh định lý này.

Bây giờ xét tổng số lần mỗi phần tử được giữ cố định bởi các hoán vị. Đặt

$$S_{ij} = \begin{cases} 0, & a_i \notin Z_j, \text{ nghĩa là } j \text{ bị thay đổi dưới } a_i, \\ 1, & a_i \in Z_j, \text{ nghĩa là } j \text{ không đổi dưới } a_i. \end{cases}$$

Ký hiệu $D(a_i)$ là số phần tử không đổi dưới hoán vị a_i . Bảng tổng quát có dạng:

hoán vị	1	2	3	...	16	$D(a_i)$
a_1	$S_{1,1}$	$S_{1,2}$	$S_{1,3}$	...	$S_{1,16}$	$D(a_1)$
a_2	$S_{2,1}$	$S_{2,2}$	$S_{2,3}$	...	$S_{2,16}$	$D(a_2)$
a_3	$S_{3,1}$	$S_{3,2}$	$S_{3,3}$	...	$S_{3,16}$	$D(a_3)$
a_4	$S_{4,1}$	$S_{4,2}$	$S_{4,3}$	...	$S_{4,16}$	$D(a_4)$
$ Z_j $	$ Z_1 $	$ Z_2 $	$ Z_3 $	...	$ Z_{16} $	

Ví dụ: phương án 1 không đổi dưới a_1 , nên $S_{1,1} = 1$. Phương án 3 bị thay đổi dưới a_3 , nên $S_{3,3} = 0$. Dưới a_1 , cả 16 phương án đều không đổi, nên $D(a_1) = 16$. Dưới a_2 , chỉ có hai phương án 1 và 2 không đổi, nên $D(a_2) = 2$.

Trong trường hợp tổng quát, ta có

$$\sum_{j=1}^n |Z_j| = \sum_{i=1}^s D(a_i),$$

trong đó $s = |G|$. Xét về trái. Giả sử

$$N = \{1, \dots, n\}$$

có L lớp tương đương:

$$N = E_1 + E_2 + \dots + E_L.$$

Nếu j và k thuộc cùng một lớp tương đương thì $|Z_j| = |Z_k|$. Do đó

$$\sum_{k=1}^n |Z_k| = \sum_{i=1}^L \sum_{k \in E_i} |Z_k| = \sum_{i=1}^L |E_i| \cdot |Z_i| = L \cdot |G|.$$

Ở đây L chính là số trạng thái tổ hợp khác nhau về bản chất mà ta cần tìm. Vì vậy

$$L = \frac{1}{|G|} \sum_{k=1}^n |Z_k| = \frac{1}{|G|} \sum_{j=1}^s D(a_j).$$

Áp dụng vào ví dụ đầu, ta được

$$L = \frac{16 + 2 + 4 + 2}{4} = 6,$$

đúng với kết quả liệt kê. Công thức này gọi là **bổ đề Burnside**.

Tuy nhiên, việc tính $D(a_j)$ không hề dễ nếu làm trực tiếp. Nếu dùng tìm kiếm, độ phức tạp tổng quát là

$$\mathcal{O}(nsp),$$

trong đó n là số phần tử, s là số hoán vị, còn p là số ô. Ở đây n có thể rất lớn. Vì vậy bước tiếp theo là tìm một cách tính $D(a_j)$ đơn giản hơn.

4 Chu trình và định lý Pólya

Chu trình. Ký hiệu

$$(a_1 a_2 \cdots a_n) = \begin{pmatrix} a_1 & a_2 & \cdots & a_{n-1} & a_n \\ a_2 & a_3 & \cdots & a_n & a_1 \end{pmatrix}.$$

Đây gọi là một chu trình bậc n . Mọi hoán vị đều có thể viết thành tích của một số chu trình đôi một rời nhau. Hai chu trình $(a_1 a_2 \cdots a_n)$ và $(b_1 b_2 \cdots b_m)$ được gọi là rời nhau nếu

$$a_i \neq b_j$$

với mọi i, j . Ví dụ:

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 3 & 5 & 1 & 4 & 2 \end{pmatrix} = (13)(25)(4).$$

Cách biểu diễn này là duy nhất nếu bỏ qua thứ tự viết các chu trình. Số chu trình của hoán vị là số chu trình xuất hiện trong biểu diễn đó. Chẳng hạn $(13)(25)(4)$ có 3 chu trình.

Với các khái niệm trên, ta đổi góc nhìn cho bài toán 2×2 . Đánh số bốn ô của bảng như sau:

2	1
3	4

Xây dựng nhóm hoán vị

$$G' = \{g_1, g_2, g_3, g_4\}, \quad |G'| = 4,$$

và gọi $c(g_i)$ là số chu trình của g_i . Dưới tác động của G' , ta có:

phép quay	dạng chu trình	$c(g_i)$
$g_1 : 0^\circ$	$(1)(2)(3)(4)$	4
$g_2 : 90^\circ$	$(4\ 3\ 2\ 1)$	1
$g_3 : 180^\circ$	$(1\ 3)(2\ 4)$	2
$g_4 : 270^\circ$	$(1\ 2\ 3\ 4)$	1

Ta nhận thấy: số hình nhận được khi các đối tượng trong cùng một chu trình của g_i phải tô cùng màu là $m^{c(g_i)}$, và con số này đúng bằng số hình không đổi dưới hoán vị a_i trong nhóm G . Với $m = 2$:

$$2^{c(g_1)} = 2^4 = 16 = D(a_1),$$

$$2^{c(g_2)} = 2^1 = 2 = D(a_2),$$

$$2^{c(g_3)} = 2^2 = 4 = D(a_3),$$

$$2^{c(g_4)} = 2^1 = 2 = D(a_4).$$

Từ đó rút ra kết luận sau.

Định lý Pólya. Giả sử $G = \{g_1, \dots, g_s\}$ là một nhóm hoán vị trên p đối tượng. Tô p đối tượng bằng m màu. Khi đó số phương án tô màu khác nhau về bản chất là

$$L = \frac{1}{|G|} (m^{c(g_1)} + m^{c(g_2)} + \dots + m^{c(g_s)}),$$

trong đó $c(g_i)$ là số chu trình của hoán vị g_i .

Đây chính là định lý Pólya. Nếu dùng định lý Pólya, độ phức tạp là

$$\mathcal{O}(sp),$$

với s là số hoán vị và p là số ô. So với cách dùng bổ đề Burnside một cách trực tiếp, đây là một cải thiện lớn. Định lý Pólya khai thác cấu trúc nội tại của đối tượng, tổng kết quy luật, loại bỏ nhiều tìm kiếm mù quáng không cần thiết, và hạ độ phức tạp của lớp bài toán này xuống mức rất thấp.

5 Bài toán bảng vuông tổng quát

Bây giờ đổi bài toán thành: với bảng $n \times n$, mỗi ô có thể tô một trong m màu, hãy tính số phương án khác nhau về bản chất dưới các phép quay.

5.1 Phân tích bằng tìm kiếm cải tiến

Trước hết xét cách tìm kiếm dễ nghĩ tới nhất, được đưa trong phụ lục. Cách tìm kiếm đó có hiệu quả rất thấp và còn nhiều chỗ có thể cải tiến. Trước đó ta đã dùng cách “tìm rồi kiểm tra”, nên tính mù quáng rất cao. Điều mong muốn hơn là vừa tìm vừa kiểm tra, tránh quá nhiều liệt kê không cần thiết, thậm chí đưa điều kiện kiểm tra vào ngay biên tìm kiếm để loại bỏ hoàn toàn các liệt kê vô hiệu. Mong muốn này có thể thực hiện được.

Ta có thể chia bảng thành bốn hình chữ nhật không chồng nhau, mỗi hình có chiều dài

$$\left\lfloor \frac{n+1}{2} \right\rfloor$$

và chiều rộng

$$\left\lfloor \frac{n}{2} \right\rfloor.$$

Khi n chẵn, bốn hình chữ nhật này phủ kín bảng. Khi n lẻ, còn lại ô chính giữa; ô này không đổi dưới mọi phép quay, nên tô màu nào cũng không ảnh hưởng tới các ô khác.

Gọi m màu là $0, 1, \dots, m-1$. Trong mỗi hình chữ nhật, ta đọc màu của các ô theo một thứ tự cố định: hình chữ nhật trái trên đọc từ trái sang phải, từ trên xuống dưới; hình chữ nhật phải trên đọc từ trên xuống

dưới, từ phải sang trái; và tương tự cho hai phần còn lại. Như vậy mỗi hình chữ nhật sinh ra một số ở hệ cơ số m , rồi có thể xem như một số thập phân trong khoảng

$$0 \text{ tới } m^{\lfloor n^2/4 \rfloor} - 1,$$

vì

$$\left\lfloor \frac{n}{2} \right\rfloor \left\lfloor \frac{n+1}{2} \right\rfloor = \left\lfloor \frac{n^2}{4} \right\rfloor.$$

Nhờ đó, một bảng được giản lược thành bốn số nguyên. Trong mỗi lớp tương đương, chỉ cần giữ phương án có số ở góc trái trên lớn nhất; nếu bằng nhau thì so tiếp theo chiều kim đồng hồ. Khi liệt kê, ba số còn lại không được lớn hơn số ở góc trái trên, nên hiệu quả là tốt nhất theo cách nhìn này.

Xét kỹ hơn, khi số ở góc trái trên là i , với $0 \leq i \leq R-1$ và

$$R = m^{\lfloor n^2/4 \rfloor},$$

ta có bốn loại:

1. Ba số còn lại đều nhỏ hơn i : có i^3 phương án.
2. Số ở góc phải trên bằng i , hai số còn lại nhỏ hơn i : có i^2 phương án.
3. Số ở góc phải trên và góc phải dưới đều bằng i , số ở góc trái dưới không lớn hơn i : có $i+1$ phương án.
4. Số ở góc phải dưới bằng i , hai số còn lại nhỏ hơn i , và số ở góc phải trên không nhỏ hơn số ở góc trái dưới: có $i(i+1)/2$ phương án.

Do đó

$$\begin{aligned} L &= \sum_{i=0}^{R-1} \left(i^3 + i^2 + i + 1 + \frac{1}{2}i(i+1) \right) \\ &= \sum_{i=0}^{R-1} \left(i^3 + \frac{3}{2}i^2 + \frac{3}{2}i + 1 \right) \\ &= \sum_{i=1}^R \left((i-1)^3 + \frac{3}{2}(i-1)^2 + \frac{3}{2}(i-1) + 1 \right) \\ &= \sum_{i=1}^R \left(i^3 - \frac{3}{2}i^2 + \frac{3}{2}i \right) \\ &= \frac{1}{4}R^2(R+1)^2 - \frac{3}{2} \cdot \frac{1}{6}R(R+1)(2R+1) + \frac{3}{2} \cdot \frac{1}{2}R(R+1) \\ &= \frac{1}{4}(R^4 + R^2 + 2R). \end{aligned}$$

Khi n lẻ, còn cần nhân thêm một thừa số m cho ô trung tâm.

Như vậy ta đã khéo léo thu được một công thức. Nhưng để nghĩ ra công thức này cần nhiều trí tuệ và thời gian. Mặt khác, phương pháp này chỉ hữu dụng cho đúng bài toán này, khó áp dụng rộng cho cả một lớp bài. Vì vậy nếu nắm được một nguyên lý có phạm vi dùng rộng hơn, việc giải các bài tương tự sẽ dễ hơn nhiều.

5.2 Giải bằng định lý Pólya

Ta dùng định lý Pólya theo ba bước.

Bước 1: xác định nhóm hoán vị. Ở đây rõ ràng chỉ có bốn hoán vị: quay 0° , quay 90° , quay 180° , quay 270° . Vì vậy

$$G = \{\text{quay } 0^\circ, \text{quay } 90^\circ, \text{quay } 180^\circ, \text{quay } 270^\circ\}.$$

Bước 2: tính số chu trình. Trước hết đánh số tuần tự các ô từ 1 tới n^2 , sau đó dùng một mảng hai chiều ghi trạng thái sau phép hoán vị. Cuối cùng, tìm kiếm trên các ô để tính số chu trình của từng phép quay. Độ phức tạp là tuyến tính theo số ô.

Bước 3: thay vào công thức. Dùng trực tiếp định lý Pólya:

$$L = \frac{1}{|G|} (m^{c(g_1)} + m^{c(g_2)} + \dots + m^{c(g_s)}).$$

Chương trình minh họa. Bản gốc dùng Pascal. Dưới đây giữ nguyên cấu trúc chương trình, dịch chú thích sang tiếng Việt không dấu.

```
const
  maxn=10;
var
  a,b:array[1..maxn,1..maxn] of integer; {ghi trạng thái bang}
  i,j,m,n:integer; {m là số màu; n là kích thước bang}
  l,l1:longint;

procedure xz; {quay bang 90 độ}
var
  i,j:integer;
begin
  for i:=1 to n do
    for j:=1 to n do
      a[j,n+1-i]:=b[i,j];
  b:=a
end;

procedure xhj; {tính số chu trình của trạng thái hiện tại}
var
  i,j,il,jl,k,p:integer;
begin
  k:=0; {đếm số chu trình}
  for i:=1 to n do
    for j:=1 to n do
      if a[i,j]>0 then {tìm ô chưa tham}
      begin
        inc(k); {thêm một chu trình}
        il:=(a[i,j]-1) div n;
        jl:=(a[i,j]-1) mod n+1; {ô kế tiếp trong chu trình}

        a[i,j]:=0; {danh dấu đã tham}
        while a[il,jl]>0 do begin
          p:=a[il,jl]; {lưu thông tin ô hiện tại}
          a[il,jl]:=0; {danh dấu đã tham}
          il:=(p-1) div n+1;
          jl:=(p-1) mod n+1 {ô kế tiếp trong chu trình}
        end {kết thúc khi đã tham trọn chu trình}
```

```

        end;
    l1:=1;
    for i:=1 to k do l1:=l1*m; {tinh m mu k}
    l:=l+l1 {cong vao tong}
end;

begin
    writeln('Input m,n=');
    readln(m,n); {nhap du lieu}
    for i:=1 to n do
        for j:=1 to n do
            a[i,j]:=(i-1)*n+j; {khởi tạo trạng thái ban đầu}
        b:=a;
        xhj; {quay 0 độ}
        xz; xhj; {quay 90 độ}
        xz; xhj; {quay 180 độ}
        xz; xhj; {quay 270 độ}
        l:=l div 4;
        writeln(l); {xuất kết quả}
    readln
end.

```

Trong chương trình trên, bản gốc tạm thời tránh xử lý số lớn vì điều đó không liên quan trực tiếp tới nội dung đang trình bày.

Nếu suy nghĩ thêm, ta còn có thể tối ưu lời giải này bằng cách xét riêng các trường hợp của n :

- Khi n chẵn: dưới quay 0° , có n^2 chu trình; dưới quay 180° , có $n^2/2$ chu trình; dưới quay 90° và 270° , có $n^2/4$ chu trình.
- Khi n lẻ: dưới quay 0° , có n^2 chu trình; dưới quay 180° , có $(n^2 + 1)/2$ chu trình; dưới quay 90° và 270° , có $(n^2 + 3)/4$ chu trình.

Gộp lại:

$$c(0^\circ) = n^2, \quad c(180^\circ) = \left\lfloor \frac{n^2 + 1}{2} \right\rfloor,$$

$$c(90^\circ) = c(270^\circ) = \left\lfloor \frac{n^2 + 3}{4} \right\rfloor.$$

Vì vậy

$$L = \frac{1}{4} \left(m^{n^2} + m^{\lfloor (n^2+3)/4 \rfloor} + m^{\lfloor (n^2+1)/2 \rfloor} + m^{\lfloor (n^2+3)/4 \rfloor} \right).$$

Công thức này khớp hoàn toàn với kết quả thu được bằng phân tích tìm kiếm ở trên. Sau phần phân tích, một bài tưởng như khó đã được giải gọn; phần còn lại trong lập trình chỉ là xử lý số lớn.

Qua các ví dụ trên, có thể thấy ưu thế của nguyên lý Pólya rất rõ khi so với tìm kiếm. Nó không chỉ nâng cao mạnh hiệu quả thời gian của chương trình, mà thậm chí còn không làm tăng độ khó lập trình. Trí tuệ và kinh nghiệm rất quan trọng, nhưng nắm được các nguyên lý nền tảng giúp việc giải bài chắc chắn hơn. Vì vậy, bên cạnh giải bài, ta cũng không nên quên học các kiến thức có tính nguyên lý.

6 Phụ lục: chương trình tìm kiếm

Chương trình sau là phương pháp tìm kiếm được nhắc tới ở phần phân tích. Nó liệt kê mọi cách tô và chỉ đếm phương án lớn nhất trong mỗi lớp tương đương dưới phép quay.

```
const
  maxn=10;
type
  sqtype=array[1..maxn,1..maxn] of byte;
var
  n,total,m:integer;
  sq:sqtype;

function big:boolean; {kiem tra phuong an co lon nhat trong lop khong}
var
  units:array[2..4] of sqtype; {ba trang thai sau khi quay}
  i,j,k:integer;
begin
  for i:=1 to n do
    for j:=1 to n do begin
      units[2,j,n+1-i]:=sq[i,j];
      units[3,n+1-i,n+1-j]:=sq[i,j];
      units[4,n+1-j,i]:=sq[i,j]
    end;
  big:=false;
  for k:=2 to 4 do {so sanh}
    for i:=1 to n do begin
      j:=1;
      while (j<=n)and(sq[i,j]=units[k,i,j]) do inc(j);
      if j<=n then
        if sq[i,j]<units[k,i,j]
          then exit
          else break
    end;
  big:=true
end;

procedure make(x,y:byte); {liet ke mau cho tung o}
var i:integer;
begin
  if x>n then begin
    if big then inc(total);
    exit
  end;
  for i:=1 to m do begin
    sq[x,y]:=i;
    if y=n then make(x+1,1) else make(x,y+1)
  end
end
```

```
end;  
  
begin  
  writeln('Input m,n=');  
  readln(m,n);  
  total:=0;  
  make(1,1);  
  writeln(total);  
  readln  
end.
```